Assignment - 15

1] What is concept of function overloading
→ Refer Q.10, Assignment - 13

2] Explain diff bet" constant & non-constant function.
→ 1] Constant function:
- Function that cannot modify any non-static member variables of class.
- These are declared by adding the const keyword at the end of function signature.
- These functions are often used to ensure that the object remains unchanged when function is called.
Syntax: return type function_name(parameters) const;
Eg: int fun() const;

2] Non-constant functions
- Can modify the non-static member variables of class.
- Declared normally w/o const keyword
- Used for modifying / updating state of object.
Syntax: return-type function_name(parameters);
Eg: void fun(int a);

3] Explain concept of member initialization list of constructor in C++.
→ - Special syntax used to initialize member variable of a class.
- Way to initialize class member directly when a constructor is invoked.
- It is useful in dealing with constant member, reference members or member objects that don't have default constructors.

- A member initialization list initializes class members before constructor body execute.
- It is written after the constructors parameter list separated by a colon (:).

Syntax: ClassName (parameter) : member1 (value1), member2 (value2)

{ // Body }

Use case!

1) Const members - Initialized at time of object creat^n & this can only be done using a member initialization list.

2) Reference members - Since references must be initialised when declared they require a member initialization list.

3) Member objects w/o Default constructors: If a class member is an object of another class w/o a default constructor it must be initialised explicitly.

4) Base Class Initialization: If class inherits from a base class, the base class constructor can be called using member initialization list.

4) How can we initialize the constant characteristics of a class?

→ - Constant characteristics of a class must be initialized at time of object creat^n bcoz they cannot be modified later.

This can be achieved using - 'Constructor initializer list'

1) Const Data member:

are read only once initialised. They must be initialized explicitly when object is created bcoz they don't get a default value.

Q) Constructor Initializer List :

→ The Initializer list allows initialization of const members & other member of class before body of constructor execute.

Q) Can we use pointer in case of static member function of a class?

→ Yes, we can use pointer in case of class.
- static member function belongs to class rather than any particular object of class.
- They don't have access to non-static data members or member functions of class bcoz they are not tied to any instance. They can only access other static members of class
- A pointer can be used to reference a static member func^n since it is not associated with any object instance.

Code:

```
#include <iostream>
using namespace std;
Class Demo {
public:
    static void display () {
    cout << " static member function" << endl;
    }
};
int main() {
 void (*funcptr) () = &Demo :: display; // pointer to static
 funcptr ();
  Demo :: display (); // Direct call w/o an object
 return 0;
}
```

6] Explain what are limitations of static func^{n} of a class.

→ 1] No Access to non-static members:
Static function cannot directly access non-static member of a class

Ex. Class Demo {
int value;
public:
static void display() {
value = 10; // cannot access value
}
};

2] No 'this pointer':
Static functions don't receive 'this' pointer bcoz they are not tied to a specific object instance.

3] Limited to Static members:
Static functions can only directly work with other static data members.

4] Not Virtual:
Static member functions can't be declared as virtual
→ This is bcoz virtual functions rely on 'this' pointer to determine appropriate func^{n} to call at runtime which static function lacks.

7] How to initialize static constant characteristics of of a class?

→ - Static const data member are class level constants shared by all objects of class
- Declared: 'static const' in class
- Since they are constant their value cannot be changed after initialization,
- Must be → Defined & initialized outside class

```
#include <iostream>
using namespace std;
class Demo {
public:
static const int constantValue;
};
const int Demo :: constantValue = 42;
int main() {
cout << "static constant value:" << Demo :: constantValue << endl;
return 0;
}
```

8] Explain why constant object cannot call non-constant member function of class?

→ A const object can't call non-const member function because - doing so would violate concept of → Immutability

1] Constant object : When an object is declared as const it means that its data members cannot be modified after initialization.

Ex. const ClassName obj;

2] Non-constant member function :

By default, member functions of a class can modify data members of object.

3] Violation of immutability : If a constant object were allowed to call a non-const member function, the function could potentially modify data members of object, violating const restriction.

9] Can we call static member function of a class using object of a class?

→ Yes, static member ..... class

- A static member function is declared with static keyword.
- It can only access other static members of class.
- It cannot access non-static data members or member functions of class.
- When we call a static member function using an object of class, the compiler internally treats it as if the function is being called on class name.

```
#include <iostream>
using namespace std;
class MyClass {
public:
    static void display() { // static member func
        cout << "static function called" << endl;
    }
};
int main() {
    MyClass obj; // object creation
    obj.display(); // calling static member using object
    MyClass::display(); // using Class name
    return 0;
}
```

10] What is diff between constant input argument & non-const arguments of function with program.

Diff is lies in whether the function is allowed to modify passed argument or not.

1] Constant Input Arguments:
- When an argument is declared as const, it cannot be modified within the function.
- Used when the function only needs to read argument but not modify it.

- Ensures integrity of passed data & avoids accidental modifications.

Non-Constant Arguments :

- Non — " — can be modified within the function.
- Used when the function needs to update or change the value of argument.

```
#include <iostream>
using namespace std;
void display (const int num) {
 Cout << " Constant Argument Value: " << num << endl;
}
void increment (int num) {
  num = num + 1 ;
  cout << " modified value : " << num << endl;
}
int main () {
   int x = 10;
   display(x);
   increment(x);
   Cout << " original value " << x << endl;
   return 0;
}
```

Output : Constant Argument value : 10
Modified value : 11
original value : 10